



**UNCUYO**  
UNIVERSIDAD  
NACIONAL DE CUYO

# PARADIGMAS DE PROGRAMACIÓN

## UNIDAD 3

Dr. Pablo Javier Vidal

Facultad de Ingeniería  
Universidad Nacional de Cuyo

2026

- 1 Traductores
- 2 Tabla de Símbolos
- 3 Lenguajes de Scripting y de Dominio
- 4 Abstracción y Polimorfismo: Comparación entre lenguajes
- 5 Closures
- 6 Ejecución de programas y memoria
- 7 Sumario

# Traductores

# Traductor



# Traductor



# Proceso de ejecución

```
package main

import "fmt"

func main() {
    fmt.Printf("hello, world")
}
```

Lenguaje de alto nivel que  
entiende el programador



```
0101010111101110001101
0100010100010101001010
0101010010101010000101
0011010001010100011110
0110010100101010101001
1110001101010010010001
```

Lenguaje de máquina que  
entiende el procesador

# Conceptos Básicos

- **Traductor:** Programa que toma como entrada un texto escrito (llamado fuente) y da como salida otro texto (llamado objeto).
- **Compilador:** Traductor cuyo fuente es un lenguaje de alto nivel y cuyo objeto es un lenguaje de bajo nivel.
- **Interprete:** Compilador que ejecuta al mismo tiempo que traduce. Es un programa que no genera un programa equivalente, sino que toma una sentencia del programa fuente en un lenguaje de alto nivel y la traduce al código equivalente y al mismo tiempo lo ejecuta

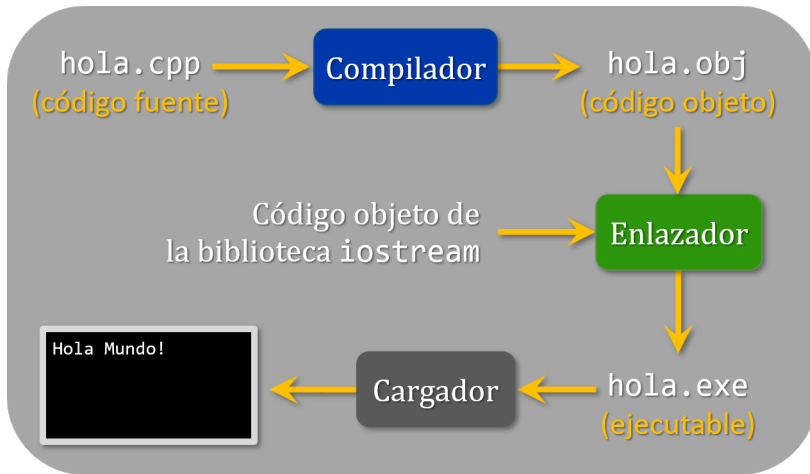


# Ventajas

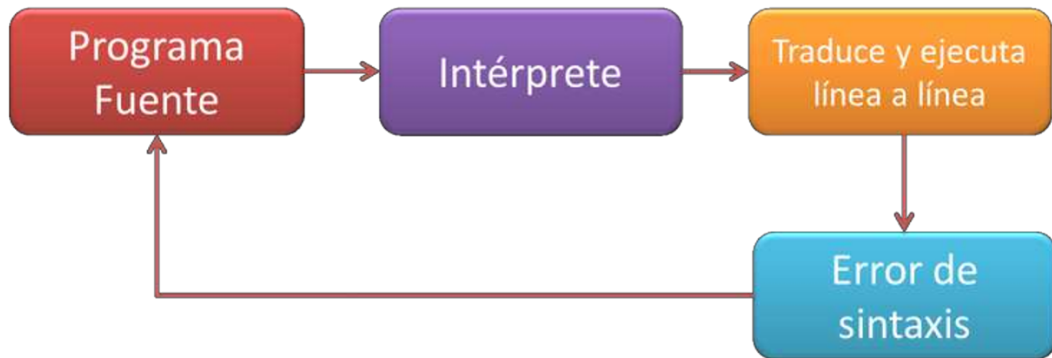
## Ventajas de los compiladores frente a los interpretes

- Solo se compilan una vez
- Ejecución más rápida
- Optimización de código e información del error más detallada.

# Proceso de compilación - ejemplo



# Interprete



# Interprete

## Programa (Sesión interactiva)

```

22
23 function compare_name(sequence a, sequence b)
24 -- Compare two sequences (records) according to NAME.
25 return compare(a[NAME], b[NAME])
26 end function
27
28 function compare_pop(sequence a, sequence b)
29 -- Compare two sequences (records) according to POPULATION.
30 -- Note: comparing b vs. a, rather than a vs. b, makes
31 -- the bigger population come first.
32 return compare(b[POPULATION], a[POPULATION])
33 end function
34
35 sequence sorted_by_pop, sorted_by_name
36 integer by_pop, by_name
37
38
39
40 sorted_by_pop = custom_sort_by_pop, statistics
41 sorted_by_name = custom_sort_by_name, statistics
42
43
44 puts(1, "sorted by population(1) sorted by name(1)")
45 for i = 1 to length(sorted_by_pop) do
46   printf(1, "NAME %15s(1) POP %15s(1)",
47     sorted_by_pop[i] & sorted_by_name[i])
48 end for
49

```

Comando actual



Resultado

## Intérprete

```

00EB: FB CB C9 06 D0 2A A9 00 22
00F0: 8D FB CB 8D FD C8 AE E3 C9
00FB: C1 A9 01 9D 7B C1 A9 E0 CA
C100: 8D FF 07 AD 7C 05 8D 81 D2
C10B: C1 20 84 C1 AD 3F 89 8D 15
C110: FB 89 AD 21 89 8D F9 89 FB

```

```

10 DATA 1C00,A9,4C,8D,05,1E
11 DATA 1C08,02,A0,0B,A2,05
12 DATA 1C10,95,C2,8B,CA,10
13 DATA 1C18,37,1C,EA,EA,EA
14 DATA 1C20,02,E6,CA,E6,C9
15 DATA 1C28,CA,20,A4,CC,20
16 DATA 1C30,00,B1,C9,AS,A6
17 DATA 1C38,CB,00,02,E6,CB

```

Estado  
Sesión

Ejecución

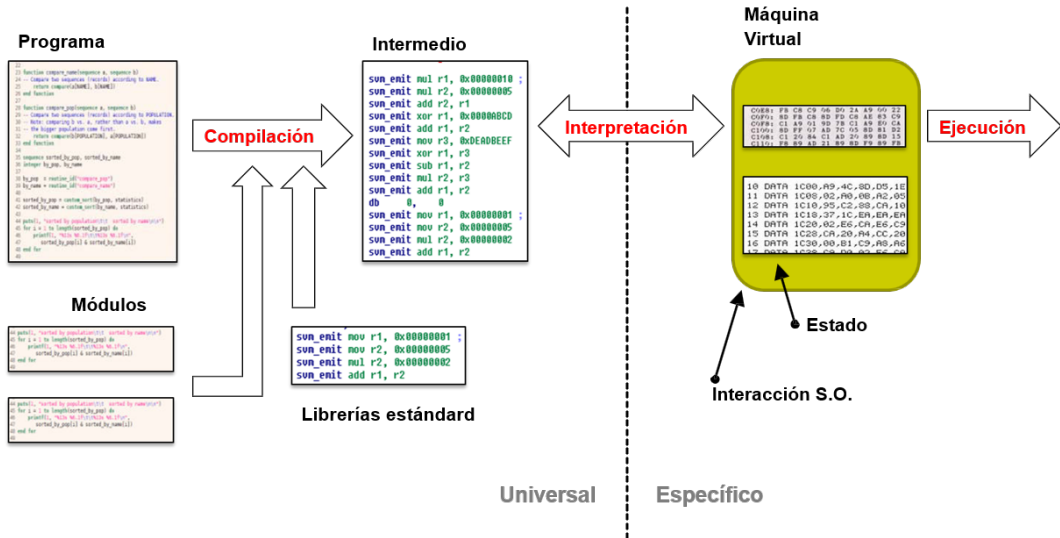
I/O

# Ventajas

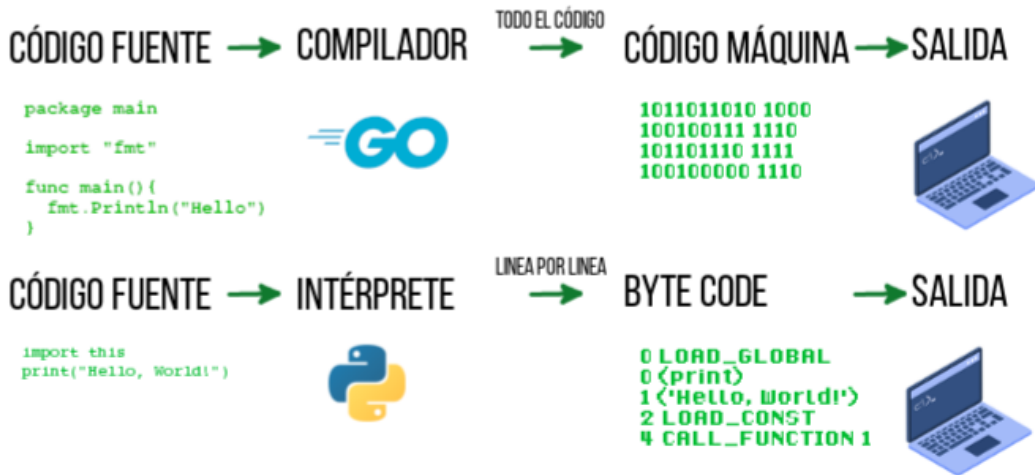
## Ventajas de los Interpretes frente a los Compiladores

- Menor coste espacial
- Mayor interactividad en desarrollo
- Añadir código *en caliente*

# Compilación e interpretación



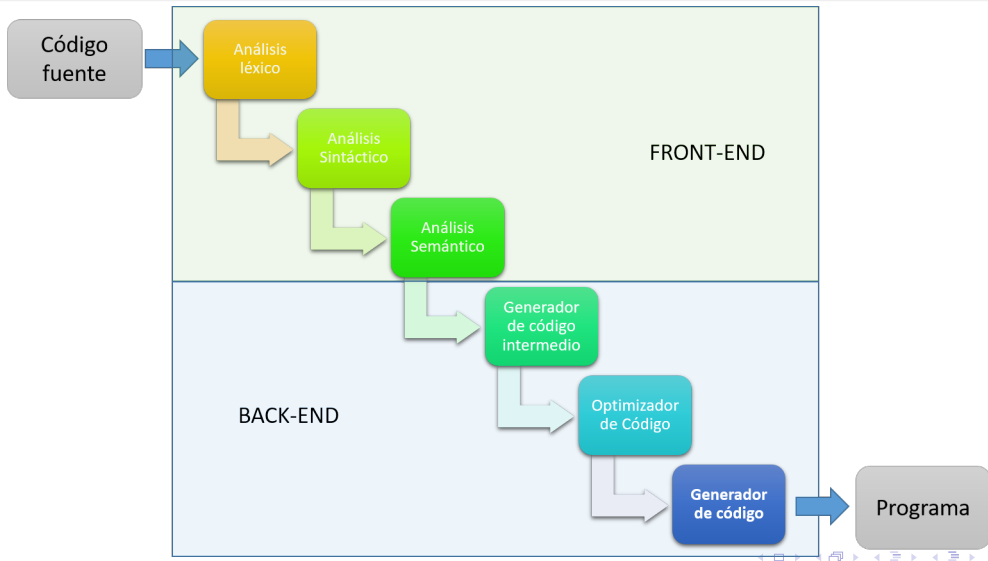
# Compilación e interpretación



# Estructura de un compilador

Para la realización del proceso de traducción es necesario dividir el compilador en varias fases.

# Estructura



# Tabla de símbolos

## Estructura

Estructura para almacenar información creada y mantenida por los compiladores acerca de la ocurrencia de diversas entidades, tales como nombres de variables, nombres de funciones, objetos, clases, interfaces, etc. tabla de símbolos se utiliza en el análisis y la síntesis de un compilador.

- Palabras reservadas
- Literales
- Símbolos predefinidos
- Símbolos definidos por el programador
- ...

## El objetivo

- Colaborar con las comprobaciones semánticas.
- Facilitar ayuda a la generación de código

# Análisis léxico

- Lee el programa fuente.
- Remueve espacios en blanco, tabulaciones, saltos de línea.
- Remueve comentarios
- Agrupa lo leído en secuencias significativas conocidas como lexemas. Y para cada lexema este analizado produce como salida un token en la forma <nombre, valor>

(nombre-token, valor-atributo)

# Análisis léxico

- Palabras reservadas: Ejemplos: IF, THEN, ELSE
- Operadores: Ejemplos: '+', '==', ':='
- Cadenas de múltiples caracteres: Ejemplos: Identificador, Constante

```
int x = 3;
int y = x + 7;
while (x != y) {
    if (x > y) {
        x = x - y;
    } else {
        y = y - x;
    }
}
```



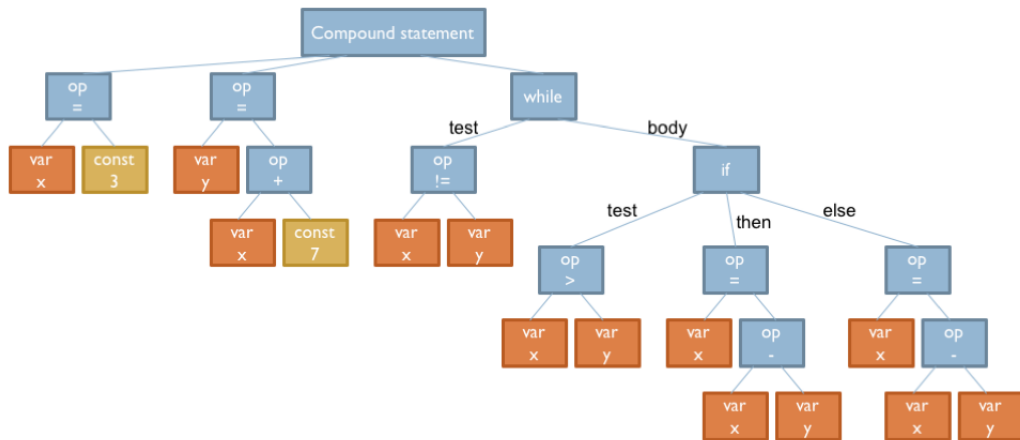
```
("int", KEYWORD)
("x", IDENTIFIER)
("=", OPERATOR)
("3", INT_CONSTANT)
(";", SPECIAL_SYMBOL)
("int", KEYWORD)
("y", IDENTIFIER)
("=", OPERATOR)
("x", IDENTIFIER)
("+", OPERATOR)
("7", INT_CONSTANT)
(";", SPECIAL_SYMBOL)
("while", KEYWORD)
("(", SPECIAL_SYMBOL)
```

...

# Análisis Sintáctico (Parser)

Es la 2da fase de compilación. Utiliza los tokens para crear una representación intermedia en forma de árbol, en la cual cada nodo interior representa una operación y los hijos del nodo representan los argumentos de la operación.





# Análisis Semántico

Utiliza el árbol sintáctico y la información en la tabla de símbolos para comprobar la consistencia semántica del programa fuente con la definición del lenguaje.

Realiza comprobación de tipos, donde verifica que cada operador tenga operandos que coincidan.

Consider:

```
int x = "bananas";
```

Syntax OK

Semantically (meaning)  
WRONG



| Var | Type |
|-----|------|
| x   | int  |

Line 1: error, invalid conversion from string constant to int

# Generador de Código Intermedio

El compilador puede crear 1 o + representaciones intermedias similares al código máquina. La representación debe ser fácil de producir y fácil de traducir en la máquina destino.

# Optimización de código

Trata de mejorar el código intermedio, de manera que produzca un mejor código destino. Mejor significa más rápido, más corto y que consuma menos recursos.

# Generación de Código

- Convierte un programa sintácticamente correcto en una serie de instrucciones a ser interpretadas por una máquina
- La máquina destino puede ser un microprocesador o una máquina abstracta tal como una máquina virtual o un lenguaje intermedio

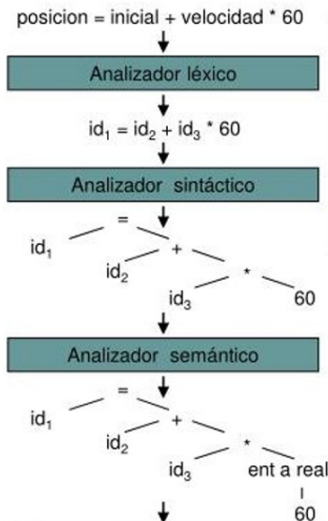
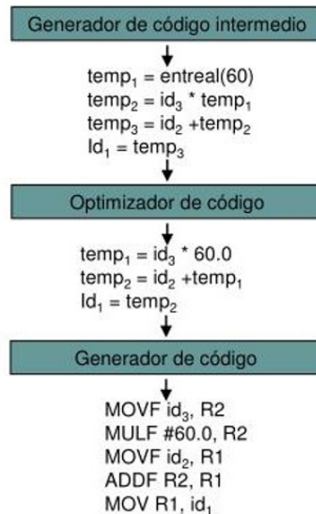


TABLA DE SIMBOLOS

|   |           |     |
|---|-----------|-----|
| 1 | posicion  | ... |
| 2 | inicial   | ... |
| 3 | velocidad | ... |
| 4 |           |     |



## Tabla de Símbolos

# ¿Qué es la tabla de símbolos?

## Definición

La tabla de símbolos es una estructura de datos utilizada por compiladores e intérpretes para almacenar información sobre los identificadores de un programa.

## Permite

- registrar variables, funciones, parámetros y tipos
- controlar el alcance (scope)
- verificar tipos
- asociar nombres con ubicaciones en memoria
- detectar errores semánticos

## Idea clave

El compilador necesita recordar información sobre cada identificador mientras analiza el programa.

# Información almacenada

Cada entrada contiene atributos:

- nombre del identificador
- categoría (variable, función, parámetro)
- tipo
- scope (global, local, bloque)
- dirección de memoria
- valor inicial
- parámetros (si es función)

Ejemplo:

Entrada en la tabla

`x → tipo:int, scope:global, dirección:0x100`

# Ejemplo base

```
int x;  
x = 5;
```

Objetivo del compilador:

- reconocer identificadores
- asociar tipos
- verificar compatibilidad

# Análisis léxico

El analizador léxico identifica tokens.

Código:

```
int x = 5;
```

Tokens:

| lexema | token     |
|--------|-----------|
| int    | KEYWORD   |
| x      | ID        |
| =      | ASSIGN    |
| 5      | NUM       |
| ;      | SEMICOLON |

En esta etapa se detectan identificadores.

# Tabla parcial tras análisis léxico

Se detectan nombres pero no siempre su tipo completo.

| nombre | categoría     |
|--------|---------------|
| x      | identificador |

El léxico normalmente:

- crea entradas preliminares
- registra lexemas

# Análisis sintáctico

El parser reconoce estructuras según la gramática.

Ejemplo de regla:

declaracion  $\rightarrow$  tipo ID ;

Cuando se reconoce la producción:

int x;

se completa la información del símbolo.

# Tabla tras análisis sintáctico

| nombre | tipo | categoría |
|--------|------|-----------|
| x      | int  | variable  |

Ahora el compilador conoce:

- tipo
- categoría

# Ejemplo incremental

```
int x;
```

```
int y;
```

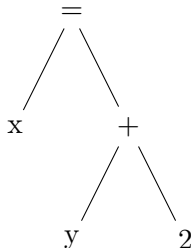
```
x = y + 2;
```

Tabla luego de declaraciones:

| nombre | tipo |
|--------|------|
| x      | int  |
| y      | int  |

# Relación con el AST

El parser genera el AST.



Durante el recorrido del AST:

- se consulta la tabla
- se verifican tipos

# Análisis semántico

Se verifican reglas del lenguaje:

- compatibilidad de tipos
- variables declaradas
- cantidad de parámetros
- visibilidad de identificadores

Ejemplo:

```
x = y + 2;
```

El compilador consulta:

- tipo(x)
- tipo(y)

# Scopes y bloques

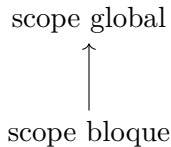
Ejemplo:

```
int x = 1;  
int x = 2;
```

Se crean múltiples tablas:

- scope global
- scope local

# Pila de tablas de símbolos



Operaciones:

- push(scope)
- pop(scope)
- lookup(nombre)

# Ejemplo con función

```
int suma(int a, int b){  
  
    int c;  
  
    c = a + b;  
  
    return c;  
}
```

# Tabla jerárquica

Scope global:

| nombre | tipo     |
|--------|----------|
| suma   | function |

Scope función:

| nombre | tipo |
|--------|------|
| a      | int  |
| b      | int  |
| c      | int  |

# Consideraciones de diseño

Aspectos importantes:

- estructura de datos eficiente
- manejo de scopes anidados
- shadowing de variables
- soporte para polimorfismo
- control de tipos
- funciones sobrecargadas

# Relación con el sistema de tipos

La tabla de símbolos permite:

- type checking
- inferencia de tipos
- coerciones
- polimorfismo

Ejemplo:

```
Animal a = new Perro();
```

tipo estático vs dinámico.

# Resumen del proceso

| etapa      | información agregada |
|------------|----------------------|
| léxico     | identificadores      |
| sintáctico | estructura           |
| semántico  | tipos y reglas       |

La tabla de símbolos conecta todas las fases del compilador.

# Idea final

La tabla de símbolos permite vincular nombres del programa con entidades del lenguaje y sus propiedades.

Sin tabla de símbolos:

- no hay verificación de tipos
- no hay control de scope
- no hay generación correcta de código

# AST: Abstract Syntax Tree

- Árbol que representa la **estructura** del programa sin detalles “superficiales” del parsing.
- Nodos típicos: `FunctionDecl`, `VarDecl`, `Assign`, `If`, `Call`, `BinOp`, `Return`.
- Facilita:
  - recorridos (visitors),
  - análisis semántico,
  - transformación (optimización),
  - generación de código.

# Resumen: ¿Qué produce un compilador? (visión por fases)

- **Análisis léxico:** tokens (identificadores, keywords, literales, operadores).
- **Análisis sintáctico:** **AST** (estructura del programa).
- **Análisis semántico:** **tabla de símbolos enriquecida** + chequeos (declaraciones, tipos, sentencias).
- (Opcional) IR / código intermedio, optimización, generación de código.

## Idea clave

El análisis sintáctico organiza la forma (estructura). El análisis semántico valida el significado (uso correcto de nombres y tipos).

# Tabla de Símbolos: ¿qué es y para qué sirve?

## Definición

Estructura que mapea **identificadores**  $\rightarrow$  **atributos** (tipo, categoría, ámbito, offset, etc.).

- Se usa para: resolución de nombres, verificación de declaraciones, chequeo de tipos, asignación de offsets, etc.
- Modela **ámbitos**:
  - global,
  - función/método,
  - bloques anidados.

## Entrada típica

`name, kind (var/func), type, scope-level, storage (global/local), offset/address.`

# Diseño típico de tabla de símbolos (ámbitos)

Dos estrategias comunes:

## 1) Pila de tablas (stack of scopes)

- Al entrar a un bloque: **push** nueva tabla.
- Al salir: **pop**.
- Búsqueda: desde el tope hacia abajo.

## 2) Árbol de ámbitos (scope tree)

- Cada ámbito tiene puntero al padre.
- Útil si se quiere mantener todo el historial (por debugging/IDE).

# ¿Qué se genera “en el sintáctico”?

## Salida principal

**AST** (o parse tree que luego se “reduce” a AST).

## Tabla de símbolos básica (posible en el sintáctico)

- Insertar **declaraciones** al reconocerlas (variables/funciones).
- Abrir/cerrar ámbitos al entrar/salir de bloques.
- **Nota:** aún puede faltar información (tipos inferidos, firmas completas, chequeos).

## Uso típico

El parser construye nodos AST y, al mismo tiempo, registra símbolos declarados.

# Análisis semántico: ¿qué valida?

- **Chequeo de declaraciones:**
  - uso antes de declarar,
  - redeclaración en el mismo ámbito,
  - visibilidad y shadowing.
- **Chequeo de tipos:**
  - compatibilidad en asignación (`int` vs `float`),
  - tipos en expresiones (`a+b`),
  - tipos y cantidad de argumentos en llamadas.
- **Chequeo de sentencias** (bien formadas):
  - `return` acorde al tipo de la función,
  - condición booleana en `if/while`,
  - control de flujo (por ejemplo, `break` dentro de loops).

# Chequeo de declaraciones: ejemplo

```
int main(void) {  
    y = 10;      // uso antes de declarar  
    int y = 3;   // declaración  
    int y = 4;   // redeclaración en el mismo ámbito  
    return y;  
}
```

## Qué detectar

- `y = 10` falla: `y` no existe en el ámbito actual.
- `int y = 4` falla: redeclaración en el mismo scope.

## Chequeo de tipos y sentencias: ejemplo

```
int f(int a) {  
    float x = 3.14;  
    int y = x;          // conversión (¿permitida? ¿warning?)  
    if (y) {            // en C: int como condición  
        return x;       // ERROR: retorna float donde se espera int  
    }  
    return a + y;  
}
```

- Tipo de `y = x`: depende de reglas del lenguaje (conversión explícita/implícita).
- Tipo de condición: en C se acepta entero como booleano; en otros lenguajes puede ser error.
- Tipo de `return`: debe coincidir con la firma `int f(int)`.

## Lenguajes de Scripting y de Dominio

# ¿Por qué “scripting”?

## Idea

Un lenguaje de **scripting** se usa típicamente para **automatizar** tareas, “pegar” componentes y orquestar herramientas.

- Frecuentes en: administración de sistemas, DevOps, web, data pipelines.
- Suelen priorizar:
  - rapidez de desarrollo,
  - flexibilidad (tipado dinámico, reflexión),
  - fuerte integración con librerías y el SO.
- Comúnmente interpretados o con runtime (aunque pueden tener JIT).

# Ejemplos de lenguajes de scripting

- **Bash**: automatización en shell, pipelines, administración.
- **Python**: scripting + propósito general (automatización, ciencia de datos).
- **JavaScript** (Node.js): scripting de servidor, tooling, automatización.
- **Lua**: scripting embebido en apps/juegos/servicios.
- **Perl/Ruby**: scripting y automatización (muy usados históricamente).

## Nota

“Scripting” describe más el **uso** y el ecosistema que una propiedad única del lenguaje.

## Ejemplo concreto: “scripting” como pegamento

### Orquestrar herramientas del SO (Bash)

```
# Contar líneas de logs y filtrar errores  
cat app.log | grep -i "error" | wc -l
```

### Automatizar un pipeline simple (Python)

```
import subprocess  
  
p = subprocess.run(["grep", "-i", "error", "app.log"],  
                   capture_output=True, text=True)  
print(len(p.stdout.splitlines()))
```

# ¿Qué es un lenguaje de dominio (DSL)?

## Definición

Un **DSL** (Domain-Specific Language) está diseñado para expresar soluciones en un **dominio particular** de forma concisa y natural.

- Ventaja: alta expresividad en el dominio (menos “ruido”).
- Desventaja: menor generalidad (no sirve para todo).
- Tipos:
  - **externos**: sintaxis propia (SQL, HTML, CSS).
  - **internos/embebidos**: se construyen dentro de un lenguaje anfitrión (por ejemplo DSLs en Haskell, Scala, Python).

# Ejemplos típicos de DSL

- **SQL**: consultas a bases de datos.
- **HTML/CSS**: descripción de interfaces web.
- **Regex**: patrones de búsqueda y validación.
- **Makefiles**: reglas de construcción.
- **LaTeX**: composición tipográfica (lenguaje de dominio para documentos).
- **WASM text format (WAT)**: representación textual de WebAssembly (orientado a un dominio/VM).

## Ejemplo concreto: DSL externo (SQL)

---

```
SELECT nombre, COUNT(*) AS n
FROM ventas
WHERE fecha >= '2026-01-01'
GROUP BY nombre
ORDER BY n DESC;
```

---

- El “dominio” es el acceso/transformación de datos relacionales.
- La sintaxis prioriza consultas declarativas, no control de flujo general.

## Ejemplo concreto: DSL embebido (mini-DSL en Python)

Idea: describir reglas (dominio: validación) con funciones

```
def rule(name, predicate):  
    return (name, predicate)  
  
rules = [  
    rule("edad>=18", lambda u: u["edad"] >= 18),  
    rule("pais=AR", lambda u: u["pais"] == "AR"),  
]  
  
user = {"edad": 20, "pais": "AR"}  
print([name for name, ok in rules if ok(user)])
```

- No es un DSL “completo”, pero muestra cómo un lenguaje anfitrión permite construir una capa de dominio.

# Scripting vs DSL: diferencias prácticas

## Scripting

- automatizar y orquestar,
- integrar herramientas,
- glue code,
- foco en productividad.

## DSL (dominio)

- expresar un dominio específico,
- alta concisión en ese dominio,
- sintaxis/semántica enfocada,
- puede ser externo o embebido.

## Observación

Un lenguaje puede ser **ambas cosas** en la práctica (ej.: Python como scripting y base para DSLs).

## Abstracción y Polimorfismo: Comparación entre lenguajes

# Objetivo de la sección

- Comparar lenguajes de programación según su **nivel de abstracción**.
- Entender cómo distintos lenguajes soportan **polimorfismo**.
- Conectar estos conceptos con **paradigmas** (imperativo, OO, funcional, lógico).

# Abstracción: idea central

**Abstraer** es ocultar detalles para trabajar con una idea más simple.

¿Qué gana el programador con más abstracción?

- Expresar soluciones **más cerca del problema** y menos del hardware.
- Reducir complejidad: **modularidad, reuso, mantenibilidad**.

¿Qué se “paga”?

- **Costo mental** (más conceptos).
- Posible **sobrecosto en ejecución** (dependiendo del lenguaje/compilador/runtime).

# Abstracción: niveles típicos (de menor a mayor)

- **Bajo nivel:** registros, memoria, saltos (ensamblador).
- **Imperativo:** variables, control de flujo, procedimientos (C).
- **Estructurado / Modular:** módulos, encapsulación (Pascal, Ada, C con disciplina).
- **OO:** clases, objetos, interfaces, herencia (Java, C#).
- **Funcional:** funciones como valores, composición, inmutabilidad (Haskell, OCaml).
- **Declarativo / Lógico:** reglas y consultas (Prolog).

## Idea clave

Más abstracción  $\Rightarrow$  menos detalle operacional explícito en el código.

# Comparación de abstracción (por paradigma)

| Paradigma  | Foco                       | Cómo se expresa la abstracción                  |
|------------|----------------------------|-------------------------------------------------|
| Imperativo | <i>Cómo hacerlo</i>        | Procedimientos, módulos, ADTs (a veces manual)  |
| OO         | <i>Objetos del dominio</i> | Clases, interfaces, encapsulación, mensajes     |
| Funcional  | <i>Transformaciones</i>    | Funciones puras, composición, tipos algebraicos |
| Lógico     | <i>Relaciones</i>          | Hechos/reglas, consultas, unificación           |

## Lectura rápida

En OO se abstrae con **entidades**; en FP con **operaciones**; en Prolog con **relaciones**.

# Polimorfismo: ¿qué significa?

**Polimorfismo** = “mismo nombre/interfaz, distintos comportamientos”.

## Pregunta guía

*¿Cómo puede un lenguaje permitir que una misma operación funcione con distintos tipos?*

- No es solo “POO”: aparece de formas distintas en varios paradigmas.
- Es clave para **reuso**, **extensibilidad** y **abstracción**.

# Tipos principales de polimorfismo

| Tipo                | Idea                  | Ejemplos típicos                                |
|---------------------|-----------------------|-------------------------------------------------|
| Ad-hoc              | Se elige según tipos  | Sobrecarga de funciones/operadores (C++, Java)  |
| Subtipo (inclusión) | Un tipo “es un” otro  | Herencia / interfaces (Java, C#)                |
| Paramétrico         | “Para cualquier tipo” | Genéricos, templates (Java, Haskell)            |
| Por coerción        | Conversión implícita  | <code>int</code> a <code>float</code> (C, Java) |

## Importante

**Ad-hoc** y **subtipo** suelen resolverse en runtime o por despacho; **paramétrico** suele ser más “uniforme” y seguro.

## Ejemplo: polimorfismo ad-hoc (sobrecarga)

```
# Idea (estilo C++/Java):  
sumar(int a, int b)    -> int  
sumar(float a, float b)-> float
```

### Lectura

- Misma operación **sumar**, distintas implementaciones.
- El compilador elige cuál usar según la firma.

## Ejemplo: polimorfismo por subtipo (OO)

```
interface Figura { double area(); }  
  
class Circulo implements Figura { area() = ... }  
class Rectangulo implements Figura { area() = ... }  
  
Figura f = new Circulo(...);  
f.area(); // despacho dinámico (runtime)
```

### Lectura

- Se programa contra **una interfaz**.
- El comportamiento depende del **objeto concreto**.

## Ejemplo: polimorfismo paramétrico (genéricos)

# Idea:

```
lista<T>           // lista para cualquier T  
map(f, lista<T>)  // aplica f a cada elemento, sin cambiar el "esqueleto"
```

### Lectura

- El algoritmo no cambia: cambia el tipo de datos.
- Suele mejorar el reuso sin perder seguridad de tipos.

## Ejemplo: polimorfismo en Prolog (relacional)

```
# "pertenece(X, L)" funciona para distintos "roles" de X
pertenece(X, [X|_]).
pertenece(X, [_|T]) :- pertenece(X, T).
```

# Se usa como:

```
pertenece(3, [1,2,3]).      % consulta booleana
pertenece(X, [1,2,3]).      % genera soluciones: X=1; X=2; X=3
```

### Lectura

- No es polimorfismo por tipos, sino por **modo de uso** (consulta/generación).
- La abstracción está en la **relación** definida.

# Comparación: abstracción vs polimorfismo

| Lenguaje/Paradigma  | Abstracción fuerte          | Polimorfismo fuerte                                 |
|---------------------|-----------------------------|-----------------------------------------------------|
| C (imperativo)      | Media (funciones, structs)  | Limitado (coerción, manual con <code>void*</code> ) |
| Java / C# (OO)      | Alta (clases, interfaces)   | Subtipo + genéricos + sobrecarga                    |
| Haskell (funcional) | Muy alta (funciones, tipos) | Paramétrico + typeclasses (ad-hoc controlado)       |
| Prolog (lógico)     | Alta (reglas/consultas)     | “Relacional” por modos/unificación                  |

## Idea de examen

*Dado un problema, justificar qué lenguaje/paradigma ofrece una abstracción más directa y qué forma de polimorfismo facilita el reuso.*

# Criterios para comparar lenguajes (guía rápida)

- **Abstracción:** ¿qué tan cerca del dominio se puede programar?
- **Seguridad de tipos:** ¿detecta errores antes de ejecutar?
- **Polimorfismo:** ¿cómo habilita reuso sin duplicar código?
- **Mecanismo:** ¿compile-time, runtime, o ambos?
- **Costo:** complejidad del lenguaje vs beneficios en diseño.

# Cierre

## Resumen

- La **abstracción** determina qué tan “alto nivel” es el pensamiento que permite el lenguaje.
- El **polimorfismo** es un motor de reuso: cambia su forma según el paradigma.
- Comparar lenguajes no es “mejor/peor”: es **adecuación** al problema y al equipo.

# C: Abstracción + “polimorfismo” manual (function pointers)

```
#include <stdio.h>

typedef struct Figura Figura;

struct Figura {
    void *obj;                // dato concreto (sin tipos)
    double (*area)(void *obj); // "método" polimórfico
};

typedef struct { double r; } Circulo;
double area_circulo(void *o) {
    Circulo *c = (Circulo*)o;
    return 3.14159 * c->r * c->r;
}

typedef struct { double w, h; } Rectangulo;
double area_rect(void *o) {
```

# Java: Subtipo + despacho dinámico (polimorfismo OO)

```
interface Figura {  
    double area();  
}
```

```
class Circulo implements Figura {  
    private final double r;  
    Circulo(double r) { this.r = r; }  
    public double area() { return Math.PI * r * r; }  
}
```

```
class Rectangulo implements Figura {  
    private final double w, h;  
    Rectangulo(double w, double h) { this.w = w; this.h = h; }  
    public double area() { return w * h; }  
}
```

```
class Demo {
```

# Haskell: Polimorfismo paramétrico + typeclasses (ad-hoc controlado)

```
data Circulo      = Circulo Double
data Rectangulo   = Rectangulo Double Double
```

```
class Figura a where
  area :: a -> Double
```

```
instance Figura Circulo where
  area (Circulo r) = pi * r * r
```

```
instance Figura Rectangulo where
  area (Rectangulo w h) = w * h
```

```
imprimirArea :: Figura a => a -> IO ()
imprimirArea f = print (area f)
```

```
main :: IO ()
main = do
```

# Prolog: Abstracción relacional (mismo predicado, distintos “modos”)

---

```
% area(Figura, A) define una relación (no "método").
```

```
area(circulo(R), A) :-
```

```
    A is 3.14159 * R * R.
```

```
area(rectangulo(W,H), A) :-
```

```
    A is W * H.
```

```
% "polimorfismo": mismo predicado, distintas formas de figura.
```

```
demo :-
```

```
    area(circulo(2.0), A1),
```

```
    area(rectangulo(3.0,4.0), A2),
```

```
    write(A1), nl,
```

```
    write(A2), nl.
```

---

## Qué muestra

Abstracción declarativa: definís **la relación** `area/2`. “Polimorfismo” por **pattern**

# Lectura comparativa (en una frase por lenguaje)

- **C**: interfaz manual + punteros a función (*rápido, flexible, poco seguro*).
- **Java**: interfaz OO + despacho dinámico (*claro para modelar dominio*).
- **Haskell**: typeclasses (*polimorfismo fuerte, chequeo estático*).
- **Prolog**: relación `area/2` (*declarativo: reglas sobre términos*).

# Closures

## Ejecución de programas y memoria

# De un programa escrito a un programa en ejecución

- Un programa fuente no se ejecuta “tal cual”: antes se analiza y traduce.
- Durante ese proceso aparecen estructuras conceptuales importantes:
  - **Tabla de símbolos:** registra nombres y atributos.
  - **AST** (*Abstract Syntax Tree*): representa la estructura del programa.
- Luego, al ejecutarse, el programa utiliza distintas regiones de memoria:
  - **segmento de código**
  - **datos globales/estáticos**
  - **stack**
  - **heap**
- Esto conecta directamente con atributos semánticos ya vistos: **nombre, dirección, tipo, valor, tiempo de vida y ámbito.**

# Vista general de memoria de un proceso

Segmento de código / instrucciones

Datos globales y estáticos

Heap  
memoria dinámica (`malloc`, `free`)

Stack  
activaciones de funciones, parámetros y variables locales

## Idea general:

- **Código:** instrucciones del programa
- **Datos:** globales y `static`
- **Stack:** crece y disminuye con llamadas
- **Heap:** reserva y liberación explícita

En C, las variables globales viven durante todo el programa; las locales viven mientras su función está activa; y el heap permite crear datos cuyo tiempo de vida puede superar una llamada puntual.

# Tabla de símbolos: qué guarda el compilador

- La **tabla de símbolos** asocia un nombre con sus atributos semánticos.
- Para cada identificador puede registrar, por ejemplo:
  - nombre
  - categoría: variable, función, parámetro
  - tipo
  - ámbito
  - nivel de bloque
  - dirección relativa / desplazamiento
  - tiempo de vida esperado
- Es clave para:
  - detectar variables no declaradas
  - resolver *shadowing*
  - chequear tipos
  - preparar la generación de código

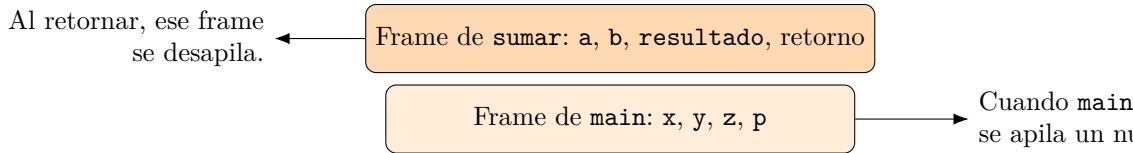
# Ejemplo de programa en C

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int g = 100;           // dato global: segmento de datos
5  static int sg = 200;  // dato estático: segmento de datos
6
7  int sumar(int a, int b) {
8      int resultado = a + b;  // variable local: stack
9      return resultado;
10 }
11
12 int main(void) {
13     int x = 10;             // local en stack
14     int y = 20;             // local en stack
15     int z = sumar(x, y);    // nueva activación en stack
16
17     int *p = malloc(sizeof(int)); // p está en stack, *p en heap
18     if (p == NULL) {
19         return 1;
20     }
21
22     *p = z * 2;
```

# Lectura semántica del ejemplo

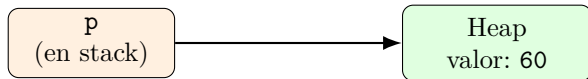
- `g` y `sg`:
  - nombre global
  - vida: todo el programa
  - ubicación: segmento de datos
- `x`, `y`, `z`, `p` en `main`:
  - variables locales
  - vida: activación de `main`
  - ubicación: stack
- `a`, `b`, `resultado` en `sumar`:
  - existen solo mientras `sumar` está ejecutándose
- `*p`:
  - dato almacenado en heap
  - su vida depende de cuándo se haga `free(p)`

# Stack en tiempo de ejecución



- El **stack** modela la secuencia de llamadas.
- Cada activación guarda parámetros, variables locales y dirección de retorno.
- Por eso una variable local deja de existir cuando termina la función.

# Heap y punteros



- El puntero `p` es una variable local: vive en el stack.
- El entero reservado con `malloc` vive en el heap.
- `free(p)` libera esa memoria dinámica.
- Si se pierde la referencia sin liberar, aparece una **fuga de memoria**.

# Shadowing y alcance léxico en C

```
1  #include <stdio.h>
2
3  int main(void) {
4      int x = 10;
5      printf("x externo = %d\n", x);
6
7      {
8          int x = 99;    // shadowing
9          printf("x interno = %d\n", x);
10     }
11
12     printf("x externo otra vez = %d\n", x);
13     return 0;
14 }
```

- El `x` interno **oculta** al `x` externo dentro del bloque.
- Esto se resuelve por **alcance léxico**: importa la estructura textual del

## Tabla de símbolos simplificada para el ejemplo

| Nombre    | Clase             | Tipo         | Ámbito         | Ubicación |
|-----------|-------------------|--------------|----------------|-----------|
| g         | variable global   | int          | global         | datos     |
| sg        | variable estática | int          | global         | datos     |
| sumar     | función           | int(int,int) | global         | código    |
| a         | parámetro         | int          | sumar          | stack     |
| b         | parámetro         | int          | sumar          | stack     |
| resultado | variable local    | int          | sumar          | stack     |
| x         | variable local    | int          | main           | stack     |
| y         | variable local    | int          | main           | stack     |
| z         | variable local    | int          | main           | stack     |
| p         | variable local    | int*         | main           | stack     |
| x         | variable local    | int          | bloque interno | stack     |

La tabla de símbolos distingue incluso nombres repetidos cuando pertenecen a ámbitos distintos.

# Tiempo de vida vs ámbito

## Ámbito

- Es textual.
- Indica dónde un nombre es visible.
- Se relaciona con bloques y funciones.

## Tiempo de vida

- Es dinámico.
- Indica cuándo una entidad existe en memoria.
- Depende de la ejecución.

### Idea clave

Una variable puede tener un ámbito bien definido en el texto del programa, pero su existencia real en memoria depende de cuándo se crea y destruye durante la ejecución.

# Errores frecuentes relacionados con memoria

- Usar una variable no inicializada
- Devolver la dirección de una variable local
- Olvidar hacer **free** → fuga de memoria
- Usar memoria ya liberada (*use-after-free*)
- Acceder fuera de rango en arreglos
- Confundir stack y heap

## Idea didáctica

Muchos errores difíciles de depurar aparecen cuando no se distingue claramente: qué nombre refiere a qué entidad, en qué región de memoria está, y cuánto tiempo vive.

# Relación con compiladores e interpretación semántica

- El análisis sintáctico construye un **AST**.
- El análisis semántico usa la **tabla de símbolos**.
- Con eso puede verificar:
  - declaraciones válidas
  - usos correctos de nombres
  - tipos compatibles
  - ámbitos y sombras
- Finalmente, en ejecución, esas decisiones se reflejan en memoria:
  - qué vive en stack
  - qué vive en heap
  - qué vive durante todo el programa

# Cierre

## Resumen

Entender cómo funciona la memoria permite conectar tres niveles:

- 1 el **texto del programa** (bloques, funciones, nombres),
  - 2 la **semántica** (tabla de símbolos, tipos, ligaduras, ámbito),
  - 3 y la **ejecución** (stack, heap, datos globales, tiempo de vida).
- En C esto es especialmente visible por el uso de direcciones y punteros.
  - Por eso C es un muy buen lenguaje para introducir la relación entre programa y memoria.

## Sumario

# Sumario

En resumen planteamos tres clasificaciones:

- En cuanto a la forma de chequeo puede ser estático/compile time, dinámico/runtime o nada.
- En cuanto a la forma de especificar el tipo de algo puede ser debil o fuerte.
- En cuanto a la forma de constituir un tipo puede ser nominal o estructural.

# Variable

- Valor: es el contenido de la celda o celdas asociadas a la variable.
- Tipo: determina el rango de valores para la variable y las operaciones que están definidas para los valores de ese tipo.
- Ligadura del tipo: antes de que una variable pueda ser utilizada debe estar ligada a un tipo. Es relevante entonces, decidir dos aspectos:
  - 1 Como se especifica el tipo
  - 2 Cuando la ligadura toma lugar